

Theseus Architecture

Theseus — research and investment firm

Document version: bound to git SHA 0034929 (2026-05-14)

Status: descriptive. This document records the system as it stands *after* Round 17 (the methodology-implementation push) and Round 18 (the consolidation and research-run pass). It is not a roadmap. The canonical version of this document is the Markdown file `docs/architecture/Theseus_Archite`; this PDF is the distribution / printing variant and may lag the Markdown by a build.

This is the connecting document for the eight scattered architecture notes the firm accumulated through Round 17/18 — `Schema_Audit_Round18.md`, `API_Envelope_Contract.md`, `Type_Boundaries.md`, `Noosphere_Module_Map.md`, `Naming_Conventions.md`, `Configuration.md`, `Trace_Coverage.md`, and `Mobile_Polish_Survey.md`. Each audits one slice of the system; this file names how the slices fit together and does not duplicate their prose.

Status markers: **[production]** — module, route or migration, test, and operational path all exist; **[experimental]** — code and tests exist but the surface is founder-only, partially instrumented, or still being calibrated; **[proposed]** — named in a companion doc as a target, not yet built.

1 System mission

Theseus is a research and investment firm building software for recorded, inspectable reasoning. The durable object the system protects is not a conclusion but a *reviewable record*: the source material, the claims pulled from it, the method that produced each claim, the strongest objection, the confidence, the revision condition, and — later — the outcome.

Three levels of work are kept distinct (see `THE_META_METHOD.md`): object-level claims (what the firm believes), methods (the reasoning procedures that produced them), and meta-methods (criteria for judging whether a method is reliable in the domain where it was used). Every architectural decision below serves one constraint: reasoning only compounds if a later reviewer can inspect the record. The firm deliberately separates working software from research ambition, and this document marks each subsystem's real status rather than its aspiration.

See also: `README.md`, `METHODOLOGICAL_REORIENTATION.md`, `THE_META_METHOD.md`.

2 Top-level components and their boundaries

Theseus is three software surfaces over one shared database.

- **Noosphere [production]** — the Python processing engine. Ingests transcripts and writings, extracts claims and source structure, builds and scores methodology profiles, computes embed-

dings, runs the coherence / adversarial / calibration tooling, drives Currents and forecasts, and writes back to the Codex database. Entry point: `python -m noosphere`.

- **Theseus Codex [production]** — the Next.js 16 / React 19 / Prisma 7 web application; both the public site and the founder control plane. Codex owns the database schema.
- **Dialectic [production]** — the PyQt6 desktop live-conversation analyzer. Captures a discussion, transcribes it with **faster-whisper**, shows argumentative signals live, and emits artifacts into the Codex upload contract. Its output is an input to Noosphere, never a peer of it.

Boundary rules. Noosphere never queries Prisma directly; it reaches the database through the `io` layer (`codex_bridge`) using generated Pydantic models. Codex never imports Python; it calls Noosphere through the FastAPI services or by reading rows Noosphere has written. Dialectic talks only to the Codex upload/auth contract. JSON Schema (`Type_Boundaries.md`) plus the unified API envelope (`API_Envelope_Contract.md`) keep the three honest at every wire crossing. The full package and route rosters are in Appendix A.

See also: `Noosphere_Module_Map.md`, `Type_Boundaries.md`, `API_Envelope_Contract.md`, `README.md`.

3 The data substrate

Theseus persists state in three places, each with a different durability and ownership rule.

```
flowchart LR
    Postgres["Postgres (Codex DB, Supabase): Conclusion / Claim / Upload,  
Round-17 method & calibration models, Span / AlertRule /  
AlertEvent, PublicationSignature"]
    Filesystem["Filesystem (THESEUS_DATA_DIR): graph.json cascade snapshot,  
embeddings/*.numpy, registries, synthesis output"]
    Manifest["Embeddings manifest: manifest.json (model, dims, checksums)"]
    Noosphere
    Noosphere --codex_bridge--> Postgres
    Noosphere --storage_client--> Filesystem
    embeddings["embeddings/*.numpy"] --- Manifest
    Codex --Prisma client--> Postgres
    Codex ..reads..> Manifest
```

Postgres is the system of record for everything relational. The schema is owned by Prisma; `Schema_Audit_Round18.md` is the table-by-table inventory. **The filesystem** (`THESEUS_DATA_DIR`) holds what does not belong in a row: the cascade snapshot, embedding arrays, registries, synthesis output, abstracted by `storage_client`. **The embeddings manifest** is the contract between the two: model, dimensionality, and per-file checksums, so Codex can detect a stale or model-mismatched embedding set without loading the arrays.

Persistence rules. Mutable rows carry `createdAt` and `updatedAt`; append-only ledger rows carry only `createdAt` — the absence of `updatedAt` is the signal a row is immutable. Every tenant-scoped row carries `organizationId` with a `Restrict` delete policy; observability tables are intentionally tenant-light. Migrations are forward-only and idempotent. Backups bundle the SQLite file, the data directory, and the manifest together so a restore is internally consistent.

See also: `Schema_Audit_Round18.md`, `Configuration.md`, `docs/operations/Migration_Runbook.md`.

4 The methodological substrate

This is the layer that distinguishes Theseus from a document store. It lives mostly in the Noosphere methods, evaluation, coherence, cascade, inquiry, and temporal packages.

- **Methods [production]** — every reasoning procedure is a registered package carrying `method.py`, `RATIONALE.md`, and `FAILURES.yaml`; snapshots are content-addressed into `MethodVersion`.
- **MQS — Methodology Quality Score [production]** — the five working criteria (progressivity, severity, aim-method fit, compressibility, domain sensitivity) plus a composite, computed in `noosphere/evaluation/mqs.py`, persisted 1:1 against each `Conclusion`. Formal definition: `docs/methods/MQS_Specification.md`.
- **Criteria tooling [production / experimental]** — the aim-method-fit and severity rubrics are production scoring paths; their ongoing calibration is tracked separately.
- **Cascade [production]** — the firm’s belief graph: truth-valued nodes joined by signed, weighted relation edges.
- **Revision [production]** — computes the blast radius of new evidence; every committed revision writes an immutable `RevisionEvent`, so replay-by-ledger can reconstruct any prior belief state.
- **Lineage [production]** — assembles and diffs the per-conclusion lineage (source → claim → method → objection → revision).
- **Bayesian belief layer (BN) [experimental]** — a derived view, not a replacement for the cascade: projects a cascade snapshot onto an acyclic Bayesian network for marginal probabilities and evidence updates. Founder-only, rebuilt on demand, holds no state.
- **Cross-domain transfer [experimental]** — packages a method for a new domain and runs a transfer-fitness study; the current output is a research study, not a production gate.

See also: `docs/methods/MQS_Specification.md`, `docs/methods/Bayesian_Belief_Layer.md`, `docs/methods/Aim`, `docs/methods/Method_Retirement_Criteria.md`, `THE_META_METHOD.md`.

5 The reasoning workflow, end to end

```
flowchart TD
    Ingestion["Ingestion (transcripts, documents, Currents)"]
    Ingestion --> Claim["-> Claim & conclusion extraction"]
    Claim --> Methodology["-> Methodology profile + MQS scoring"]
    Methodology --> Cascade["-> Cascade graph (signed weighted edges)"]
    Cascade --> Review["-> Review (coherence, adversarial, peer-review, citation chain)"]
    Review --> Objection["-> blocking objection or drift?"]
    Objection -- yes --> RevisionEngine["Revision engine"]
    RevisionEngine --> RevisionEvent["RevisionEvent ledger"]
    RevisionEvent --> Cascade
    Objection -- no --> Publication["Publication & signing pipeline"]
    Publication --> Outcome["-> Outcome (forecast resolution, calibration)"]
    Outcome --> MethodTrack["-> Method track record + drift detector"]
    MethodTrack --> RevisionEngine
```

The steps are: ingestion (uploads or `CurrentEvent`, with relevant-text filtering); claim and conclusion extraction; methodology profile generation and MQS scoring; cascade integration; review by the six-layer coherence engine, the multi-model adversarial swarm, peer review, and the citation-chain validator; revision (blast-radius preview, then a committed `RevisionEvent`); publication; and

outcome — forecasts resolve, calibration recomputes, the method track record rolls up, and the drift detector watches for a method whose calibration has decayed, which re-enters the loop at revision. The loop is deliberately a loop: an outcome is evidence, and evidence revises.

See also: `Trace_Coverage.md`, `Schema_Audit_Round18.md`, `METHODOLOGICAL_REORIENTATION.md`.

6 The publication and signing pipeline

```
flowchart LR
    ReviewedConclusion["Reviewed conclusion / article candidate"] --> CitationChain["-> Citation-chain validation"]
    CitationChain --> PublicCitable["-> public-citable sources only?"]
    PublicCitable -- no --> Held["no -> Held (founder source-triage queue)"]
    PublicCitable -- yes --> Canonicalize["yes -> Canonicalize preimage"]
    Canonicalize --> Ed25519Sign["-> Ed25519 sign (ledger/publication_signing)"]
    Ed25519Sign --> PublicationSignatureRow["-> PublicationSignature row"]
    PublicationSignatureRow --> PublicRender["-> Public render + /api/public/signature/[slug]"]
    PublicRender --> SelfCritique["-> Self-critique pass -> Addendum (dated)"]
```

Publication is gated, not automatic **[production]**. Citation gating validates every cited excerpt against the stated claim with an NLI judge; a publication blocker or a private-only source holds the piece in the founder source-triage queue. Canonicalization builds a deterministic preimage — renaming any field that participates in it invalidates every historical signature, which is why such renames are an out-of-band review item in `Naming_Conventions.md`. Signing uses an Ed25519 key held outside the web tree and writes one immutable `PublicationSignature` per published revision; any reader can re-derive and verify it. Published articles are periodically re-reviewed; a finding becomes a dated `Addendum` rather than a silent edit. Desktop-application code-signing is a separate distribution concern.

See also: `Naming_Conventions.md`, `Schema_Audit_Round18.md`, `docs/security/Threat_Model.md`.

7 The observability and operations surfaces

Spans [production / instrumentation in progress] — Round 17 added span-based observability via the `@traced` decorator; the decorator and `Span` schema are production, but only ~9% of public functions are wrapped today — `Trace_Coverage.md` is the living gap list. **Metrics and alerts [production]** — `MethodMetricRollup` aggregates per-method windows; `AlertRule` / `AlertEvent` hold threshold rules and firings. **The attention queue [production]** — a unified founder queue; every snooze/dismiss is an append-only `AttentionAction`, and schema drift at a wire boundary surfaces here rather than throwing silently. **Configuration [production]** — all environment reads route through one typed module per language, enforced by a CI gate. **Operations** — health routes, the production migration runbook, the forecast scheduler, and backup/restore are the operational surface; CI is a wall of consistency gates, including the check for this document.

See also: `Trace_Coverage.md`, `Configuration.md`, `docs/operations/Migration_Runbook.md`, `docs/operations/`

8 The public-facing surfaces and the visibility model

The Codex serves two audiences from one app, split by route group: the **founder workspace** (**authed** — uploads, dashboard, library, explorer, review, forecasts, ops, behind bcryptjs auth) and the **public site** (reviewed articles, structured responses, public Currents, forecasts, the methodology explorer, the calibration scorecard, the auto-generated research papers, critiques, revisions, and the syndication feeds). The **api** route group returns the unified envelope on every route.

The visibility model. The default is private. A conclusion, source, or citation becomes public only by an explicit gate: public article citations resolve only when the cited source is itself public; the Bayesian marginal, the raw cascade, raw transcripts, and hidden source documents are founder-only; outside-reader replies and bounty critiques are double-opt-in. The public surfaces are also held to a mobile contract — every mobile/desktop split ships both renderings and lets a CSS media query choose, so there is no post-hydration layout shift. The full route roster is in Appendix A.

See also: `Mobile_Polish_Survey.md`, `API_Envelope_Contract.md`, `Naming_Conventions.md`, `METHODOLOGICAL_RE`

9 Open questions about the architecture itself

1. **The Noosphere layering is a target, not a fact.** The hierarchical package layout is enforced by `import-linter`, but the modules have not been physically relocated and the deprecation shims are **[proposed]**.
2. **Where do cascade, cases, decisions, principles, voices live?** They are domain aggregates with no clear home in the `core/methods/inquiry` split.
3. **Observability coverage.** At ~9% `@traced` coverage the trace is not yet a reliable end-to-end picture of a request.
4. **The polymorphic citation key.** Promoting `CitationVerdict.(citationKind, citationId)` to a discriminated union is **[proposed]** and deferred.
5. **Two languages, one schema.** The JSON-Schema bridge aligns Pydantic and TypeScript, but Prisma is still a third generator; there is no single source of truth for all shared types.
6. **The BN's relationship to the cascade.** Whether a future revision engine should consume BN marginals directly is open.

See also: `Noosphere_Module_Map.md`, `Schema_Audit_Round18.md`, `Type_Boundaries.md`, `Known_Cycles.md`, `Dead_Code_Survey.md`.

Appendix A — Component index (CI-checked)

This index exists so `scripts/check_architecture_consistency.py` can confirm the document still names every Noosphere package and every top-level Codex route. The canonical, CI-checked copy of this index is in the Markdown document; this is a printed mirror.

Noosphere packages (`noosphere/noosphere/`): `core`, `ledger`, `observability`, `_generated`, `methods`, `evaluation`, `inquiry`, `coherence`, `peer_review`, `mitigations`, `rigor_gate`, `cascade`, `inference`, `cases`, `decisions`, `principles`, `voices`, `articles`, `distillation`, `temporal`, `literature`, `currents`, `forecasts`, `social`, `decay`, `io`, `extractors`, `embeddings`, `interop`, `transfer`, `external_battery`, `cli`, `cli_commands`, `docgen`, `benchmarks`.

Codex routes (theseus-codex/src/app/): home, about, ask, c, calibration, critiques, currents, forecasts, methodology, post, privacy, proof, research, revisions, login, atom.xml, feed.xml, authed, api.

Changelog

Each substantive change writes a new entry, tagged with the git SHA at which it landed.

Date	Git SHA	Change
2026-05-14	0034929	Initial architecture document. Connects the eight Round 17/18 scattered docs; describes the three components, the data and methodological substrates, the end-to-end reasoning workflow, the publication/signing pipeline, the observability and public surfaces, and the open architectural questions. Adds the CI-checked component index (Appendix A).